

Oscar Espinoza Landeta

1. DOM vs. Virtual DOM (VDOM)

Es fundamental entender que React no interactúa directamente con el DOM del navegador para cada cambio, sino que utiliza una capa intermedia para optimizar el rendimiento.

- **¿Qué es el Virtual DOM?** El VDOM es un concepto de programación donde una representación "ideal" o virtual de la interfaz de usuario (UI) se mantiene en memoria. Es una estructura de objetos ligeros en JavaScript que imitan el árbol del DOM real.
- **Proceso de Reconciliación:** React mantiene el VDOM sincronizado con el DOM "real" mediante una biblioteca como ReactDOM. Este proceso se llama **reconciliación**. Cuando el estado de un componente cambia, React actualiza el VDOM, lo compara con la versión anterior (diffing) y calcula la forma más eficiente de actualizar el DOM real.
- **Abstracción Declarativa:** Este enfoque permite la API declarativa de React: tú le dices a React *en qué estado* quieres que esté la UI, y él se encarga de manipular los atributos, manejar eventos y actualizar el DOM manualmente.
- **Diferencia con Shadow DOM:** No se debe confundir el VDOM con el Shadow DOM. El Shadow DOM es una tecnología del navegador diseñada para aislar variables y CSS en Web Components. El VDOM es un patrón implementado por bibliotecas (como React o Vue) sobre las APIs del navegador .

2. Problemas Comunes al Manipular el DOM

Manipular el DOM, ya sea manualmente (con jQuery/JS vainilla) o mediante renderizados dinámicos en frameworks, conlleva riesgos significativos para la accesibilidad si no se planifica.

- **Desincronización del Estado:** Al usar frameworks como React, se desaconseja editar elementos HTML directamente con APIs nativas (como `getElementById`), ya que esto provoca que el VDOM y el DOM real se desincronizan, causando errores de renderizado y comportamientos impredecibles.
- **Pérdida de Contexto para el Usuario:** Agregar contenido en momentos arbitrarios o sin avisar al usuario se considera un "cambio de contexto". Si el contenido aparece sin mover el foco o sin una notificación (Live Region), los usuarios de lectores de pantalla no sabrán que algo ha cambiado.

- **Destrucción de Elementos Enfocados:** Un error crítico ocurre al eliminar un elemento del DOM que actualmente tiene el foco (ej. borrar una fila de una tabla o cerrar un modal). Si no se gestiona, el foco se pierde (a menudo regresa al `body` o "a la nada"), obligando al usuario de teclado a empezar la navegación desde el principio de la página.
- **Uso de Elementos No Semánticos:** Un problema recurrente en aplicaciones React es el uso excesivo de `div` con manejadores de clic (`onClick`) en lugar de usar elementos nativos como `<button>`. Esto rompe la accesibilidad nativa del teclado y la semántica que esperan los lectores de pantalla.

3. Gestión de Foco (Focus Management)

La gestión del foco es la práctica de usar cambios programáticos del foco para guiar la atención del usuario y asegurar una navegación lógica por teclado.

- **La Regla de Oro:** Si eliminas el elemento actualmente enfocado del DOM, **debes** mover el foco a otro lugar antes o inmediatamente después de eliminarlo.
- **Cuándo Gestionar el Foco Manualmente:**
 - **Al abrir un Modal:** El foco debe moverse al interior del diálogo (generalmente al primer elemento interactivo o al contenedor del diálogo) y quedar "atrapado" allí.
 - **Al cerrar un Modal:** El foco debe regresar al elemento que disparó la apertura del modal (el botón original). Si ese botón ya no existe, debe ir al siguiente elemento lógico.
 - **Navegación en Rutas (SPA):** Al cambiar de "página" en una Single Page Application, el foco debe moverse al nuevo contenedor principal (usualmente un `H1` o un wrapper con `tabindex="-1"`) para confirmar que el contenido ha cambiado.
 - **Formularios con Errores:** Al enviar un formulario fallido, el foco debe moverse programáticamente al primer campo con error para agilizar la corrección.
- **Foco del Navegador vs. Foco del Lector de Pantalla:** El foco del navegador (`document.activeElement`) y el "cursor virtual" del lector de pantalla no siempre están en el mismo lugar. Sin embargo, mover el foco del navegador programáticamente (`element.focus()`) generalmente arrastra también el foco del lector de pantalla.
- **Técnica de Implementación:** En React y Vue, se utilizan **Refs** para acceder directamente al nodo del DOM y ejecutar el método `.focus()`. A veces es necesario usar `nextTick` o `useEffect` para asegurar que el elemento ya se ha renderizado antes de intentar enfocarlo.

4. Uso Correcto de ARIA (Accessible Rich Internet Applications)

ARIA debe usarse para complementar el HTML, no para reemplazarlo. Su mal uso puede ser peor que no usarlo en absoluto.

- **Las 5 Reglas de ARIA:**
 - **No uses ARIA** si puedes usar un elemento HTML nativo que haga lo mismo (ej. usa `<button>` en lugar de `<div role="button">`).
 - No cambies la semántica nativa a menos que sea necesario.
 - Todos los controles ARIA interactivos deben ser operables con teclado.
 - No ocultes elementos enfocables (`aria-hidden="true"`) si el usuario puede llegar a ellos con el tabulador.
 - Asegura que todos los elementos interactivos tengan un "nombre accesible" (etiqueta).
- **Regiones en Vivo (Live Regions):** Para contenido dinámico que aparece sin mover el foco (como notificaciones de chat, errores de validación en tiempo real o actualizaciones de carrito de compra), se usan regiones en vivo.
 - `aria-live="polite"`: Espera a que el usuario termine de interactuar o escuchar para anunciar el cambio (ideal para mensajes de éxito).
 - `aria-live="assertive"`: Interrumpe inmediatamente al usuario (usar solo para errores críticos).
- **Roles y Estados:** En aplicaciones complejas, es vital comunicar el estado de los componentes. Por ejemplo, un acordeón necesita `aria-expanded="true/false"` para indicar si está abierto o cerrado. React facilita la sincronización de estos atributos ARIA con el estado del componente.

Abraham Cervantes Romero

DOM y componentes dinámicos accesibles

1. DOM vs Virtual DOM

¿Qué es el DOM?

El **DOM (Document Object Model)** es la representación del contenido de una página web en forma de árbol.

Cada elemento HTML (botones, textos, inputs) es un nodo que el navegador puede leer y modificar.

Cuando cambias algo en el DOM real:

- El navegador vuelve a procesar y renderizar partes de la página

- Puede ser lento si hay muchos cambios

¿Qué es el Virtual DOM?

El **Virtual DOM** es una copia del DOM real que se mantiene en memoria (usado por librerías como React).

Funciona así:

1. El usuario interactúa
2. Se actualiza el Virtual DOM
3. React compara cambios (diff)
4. Solo actualiza lo necesario en el DOM real

Ventaja principal:

- Mejor rendimiento
- Cambios más controlados
- Menos errores visuales y de accesibilidad

2. Problemas comunes al manipular el DOM

Cuando se manipula el DOM directamente (por ejemplo con `document.getElementById`), pueden surgir problemas como:

Problemas frecuentes:

- ✗ El foco se pierde sin aviso
- ✗ Lectores de pantalla no detectan cambios
- ✗ Elementos aparecen/desaparecen sin explicación
- ✗ El usuario se confunde

Ejemplo:

Un formulario muestra un error, pero:

- El lector de pantalla no lo anuncia
- El cursor sigue en otro lugar

Resultado: mala experiencia de usuario.

3. Gestión de foco

El **foco** indica **dónde está interactuando el usuario** (teclado, lector de pantalla).

¿Por qué es importante?

- Personas que navegan con teclado dependen del foco
- Si se pierde, el usuario queda “atrapado”

Buenas prácticas:

- Mover el foco cuando aparece un modal
- Regresar el foco al cerrar un modal
- Enviar el foco al primer error de un formulario

Ejemplo correcto:

Al mostrar un mensaje de error, el foco va directo al campo incorrecto.

Esto evita confusión y frustración.

4. Uso correcto de ARIA

ARIA (Accessible Rich Internet Applications) ayuda a que los componentes dinámicos sean entendibles para tecnologías asistivas.

¿Para qué sirve ARIA?

- Explicar qué es cada elemento
- Anunciar cambios dinámicos
- Mejorar accesibilidad sin cambiar el diseño

Atributos comunes:

- aria-label: describe un elemento
- aria-live: anuncia cambios automáticos
- role: indica la función del elemento

Ejemplo:

Un mensaje de error con aria-live:

El lector de pantalla anuncia el error automáticamente

⚠ ARIA no reemplaza HTML semántico, solo lo complementa.

5. ¿Para qué sirve todo esto?

Para que la interfaz cambie sin:

✓ Romper la accesibilidad

Todos los usuarios pueden interactuar sin problemas.

✓ Confundir al usuario

Los cambios son claros y explicados.

✓ Perder foco

El usuario siempre sabe dónde está.

Lilia Hernandez Tun

DOM y componentes dinámicos accesibles

- DOM vs Virtual DOM

DOM: Es una interfaz de programación para documentos **HTML** (lenguaje de marcado de hipertexto) y documentos **XML** (lenguaje de marcado extensible). La estructura del DOM se puede representar como un árbol en la que cada rama del árbol termina en un nodo, y cada nodo contiene objetos.

Virtual

DOM:

Es una abstracción del DOM que se utiliza en aplicaciones web. En React, el Virtual DOM es una representación liviana y en memoria del DOM real. La principal funcionalidad del Virtual DOM es optimizar las actualizaciones y manipulaciones del DOM, permitiendo a React hacer cambios en la interfaz de usuario de manera más rápida y eficiente.

Característica	DOM Real	Virtual DOM (VDOM)
Definición	Representación estructural en forma de árbol del HTML en el navegador.	Representación ligera y en memoria (JavaScript) del DOM real.
Actualización	Actualiza el elemento y todos sus hijos (re-renderizado completo del nodo).	Solo actualiza los objetos que cambiaron en la copia virtual.
Rendimiento	Lento en aplicaciones grandes debido a las operaciones frecuentes de repintado.	Muy rápido; minimiza las operaciones costosas en el navegador.
Eficiencia	Menos eficiente: el navegador debe recalcular el diseño y CSS (reflow/repaint).	Muy eficiente: usa un algoritmo de "diffing" para aplicar solo los cambios mínimos necesarios.
Memoria	Consume más memoria al mantener el estado completo en el navegador.	Consume menos memoria inicial, pero requiere una librería (React, Vue) para gestionarlo.
Manipulación	Directa (ej. <code>document.createElement()</code>).	A través de estados y props del framework.

Problemas comunes al manipular el DOM

Manipular el DOM directamente (con JS "vanilla") puede causar varios problemas si no se hace con cuidado:

- **Pérdida de Estado:** Al usar `innerHTML = "` para actualizar un contenedor, borras todo lo que había dentro. Si el usuario había escrito algo en un input o seleccionado un checkbox dentro de ese contenedor, esos datos se pierden.
- **Fugas de Memoria (Memory Leaks):** Si eliminas un elemento del DOM pero olvidas eliminar los "Event Listeners" (como un click) asociados a él, esos eventos se quedan ocupando memoria en el navegador.
- **Reflows y Repaints excesivos:** Cada vez que cambias el tamaño o posición de un elemento, el navegador tiene que recalcular toda la página. Si haces esto muchas veces por segundo (como en un scroll), la página se sentirá lenta o "trabada".

Gestión de Foco (Focus Management)

La gestión de foco asegura que las personas que usan el teclado o lectores de pantalla sepan siempre en qué parte de la página están. Se refiere al control consciente de qué elemento de tu página web tiene la atención del teclado en un momento dado y cómo se transfiere ese foco entre elementos.

- El Problema del Foco Perdido: Si abres un modal dinámico y el foco se queda en el botón que lo abrió (que ahora está detrás del modal), el usuario no puede interactuar con la nueva ventana.
- Skip Links: Proporcionar enlaces ocultos que permiten al usuario "saltar" directamente al contenido principal sin tener que pasar por todo el menú de navegación cada vez.
- Focus Trapping (Atrapado de foco): En componentes como modales, el foco debe quedarse encerrado dentro del modal. Si el usuario presiona Tab y el foco se sale hacia los elementos del fondo, la accesibilidad se rompe.

Uso correcto de ARIA

Es un conjunto de atributos que mejoran la accesibilidad de contenido web, especialmente para aplicaciones dinámicas y controles personalizados. Cuando el HTML semántico no es suficiente, ARIA actúa como puente entre componentes y tecnologías asistivas.

ARIA no cambia cómo se ve la página, sino cómo se comunica con la tecnología asistida (como lectores de pantalla).

- No usar ARIA si existe un elemento HTML nativo: La regla de oro es: *"Si puedes usar un `<button>` en lugar de un `<div>` con ARIA, usa el botón"*.

Atributo / Rol	Tipo	¿Para qué sirve?	Ejemplo de uso
role="dialog"	Rol	Indica que un elemento es una ventana emergente o modal.	En el contenedor principal de tu modal.
aria-live	Estado	Avisa al lector de pantalla sobre cambios dinámicos en el contenido.	polite: Notificaciones. <u>assertive</u> : Errores.
aria-expanded	Propiedad	Indica si un elemento (como un menú) está abierto (true) o cerrado (false).	En el botón que despliega un menú.
aria-hidden	Estado	Oculto elementos al lector de pantalla que son solo visuales o están cerrados.	true: Para iconos decorativos o modales ocultos.
aria-label	Propiedad	Define un texto que etiqueta al elemento cuando no hay texto visible.	Un botón que solo tiene un icono de "X" para cerrar.
aria-controls	Relación	Indica qué elemento es controlado por el actual (vía ID).	Un botón que controla la visibilidad de un div.
aria-invalid	Estado	Indica que el valor introducido en un campo tiene un error.	Un input de formulario cuando el dato es incorrecto.

Referencias

- **Arsys.** (s.f.). *¿Qué es el DOM y cómo funciona en JavaScript?* Blog de Arsys. <https://www.arsys.es/blog/que-es-dom-javascript>
- **DevHub Educativo.** (s.f.). *Atributos ARIA para Accesibilidad Web: Guía Completa.* <https://devhub.educativo.com/atributos-aria-guia-completa>

- **Finanzas Magazine.** (s.f.). *Ventajas y desventajas del trading de DOM.* <https://finanzasmagazine.com/ventajas-y-desventajas-trading-dom>
- **Gestión de Foco (s.f.).** *Focus Management: cómo mejorar la accesibilidad y usabilidad de nuestros componentes.* DEV Community. <https://dev.to/gestion-foco-accesibilidad>
- **MDN Web Docs.** (s.f.). *¿Qué es el DOM?* Mozilla Corporation. https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model/Introduction

FRANCISCO XAVIER GIL GINEZ

DOM y componentes dinámicos accesibles

El desarrollo web moderno se apoya fuertemente en interfaces dinámicas que cambian en tiempo real según la interacción del usuario. Estas interfaces se construyen principalmente mediante la manipulación del DOM (Document Object Model) o mediante abstracciones como el Virtual DOM. Sin embargo, cuando se crean componentes dinámicos, surge un reto importante: garantizar que dichos cambios sean accesibles para todas las personas, incluidas aquellas que utilizan tecnologías de asistencia como lectores de pantalla, teclados alternativos o dispositivos adaptativos. La accesibilidad no debe considerarse un complemento, sino un requisito fundamental del diseño y la programación web.

DOM vs Virtual DOM

El DOM (Document Object Model) es una representación estructurada del documento HTML en forma de árbol. Cada elemento, atributo y texto se convierte en un nodo que puede ser consultado y modificado mediante lenguajes como JavaScript. Cuando se actualiza el DOM directamente, el navegador debe recalcular estilos, reflujo (reflow) y repintado (repaint), procesos que pueden resultar costosos en términos de rendimiento si se realizan con mucha frecuencia. Además, las modificaciones directas pueden

generar inconsistencias si no se gestionan adecuadamente los estados de los elementos interactivos.

El Virtual DOM surge como una solución a los problemas de rendimiento y complejidad del DOM tradicional. Frameworks como React, Vue o Angular emplean un árbol virtual que representa una copia ligera del DOM real. Cada vez que cambia el estado de la aplicación, se genera una nueva versión del Virtual DOM y se compara con la anterior mediante un proceso conocido como “diffing”. El sistema identifica los cambios mínimos necesarios y los aplica al DOM real de manera eficiente. Esto reduce el número de operaciones costosas y mejora la experiencia del usuario.

Desde el punto de vista de la accesibilidad, el Virtual DOM no reemplaza al DOM real, sino que actúa como intermediario. Los lectores de pantalla y otras tecnologías de asistencia interactúan únicamente con el DOM real que el navegador expone a la API de accesibilidad. Por lo tanto, aunque el Virtual DOM optimice el rendimiento, el desarrollador sigue siendo responsable de garantizar que los elementos renderizados sean semánticos, tengan roles correctos y reflejen adecuadamente los cambios dinámicos. Un error común es asumir que el uso de frameworks modernos garantiza accesibilidad automáticamente, cuando en realidad depende del uso correcto de componentes, etiquetas HTML semánticas y atributos ARIA.

Problemas comunes al manipular el DOM

Uno de los problemas más frecuentes al manipular el DOM es la pérdida de contexto para los usuarios que dependen de lectores de pantalla. Cuando un contenido cambia dinámicamente sin notificación adecuada, el usuario puede no percibir que ocurrió un cambio. Por ejemplo, si se actualiza una lista de resultados tras una búsqueda sin recargar la página, el lector de pantalla puede permanecer en el elemento anterior sin anunciar la nueva información. Esto genera confusión y rompe la experiencia de navegación.

Otro problema común es la ruptura del orden lógico del documento. Al insertar o eliminar nodos del DOM sin considerar su posición semántica, se puede alterar el flujo natural de lectura. Esto afecta tanto a usuarios de lectores de pantalla como a quienes navegan

únicamente con el teclado. Además, el uso excesivo de elementos genéricos como `div` y `span` sin roles semánticos dificulta la interpretación del contenido por parte de las tecnologías de asistencia.

También es habitual encontrar problemas relacionados con eventos JavaScript. Muchas interfaces dinámicas dependen de eventos como `onclick`, pero no contemplan eventos de teclado como `onkeydown` o `onkeypress`. Esto provoca que ciertos componentes sean inaccesibles para usuarios que no utilizan ratón. Asimismo, ocultar y mostrar contenido mediante estilos CSS sin actualizar atributos como `aria-hidden` puede hacer que elementos invisibles sigan siendo leídos por lectores de pantalla.

La manipulación directa del DOM puede provocar además inconsistencias en el estado de los componentes. Por ejemplo, un botón puede parecer deshabilitado visualmente, pero seguir siendo interactivo desde el punto de vista del lector de pantalla si no se actualiza correctamente el atributo `disabled` o el rol correspondiente. Este tipo de errores se consideran fallos críticos de accesibilidad.

Gestión de foco

La gestión del foco es un aspecto central en la accesibilidad de componentes dinámicos. El foco indica qué elemento está activo en un momento determinado y es fundamental para los usuarios que navegan mediante teclado o dispositivos asistivos. Cuando una página cambia dinámicamente, el foco debe moverse de forma lógica y predecible. Si se abre un modal, por ejemplo, el foco debería trasladarse automáticamente al primer elemento interactivo dentro del modal, evitando que el usuario siga navegando por el contenido que quedó detrás.

Un error frecuente es no controlar el foco después de acciones importantes como enviar un formulario, mostrar un mensaje de error o cargar contenido nuevo. En estos casos, el usuario puede quedar “perdido” en la interfaz sin saber que ocurrió un cambio. La buena práctica consiste en mover el foco hacia el mensaje relevante o hacia el nuevo contenido, y asegurar que este sea anunciado por el lector de pantalla.

Otro aspecto relevante es evitar trampas de foco. Esto ocurre cuando un usuario queda atrapado dentro de un componente sin posibilidad de salir utilizando el teclado. Los

diálogos, menús desplegables y carruseles son ejemplos típicos donde se debe implementar un ciclo controlado del foco (focus trap), permitiendo navegar dentro del componente pero también salir de él con una tecla como Esc.

La gestión del foco también implica mantener un orden lógico de tabulación. El uso incorrecto del atributo `tabindex` puede provocar que el foco salte de forma arbitraria entre elementos, rompiendo la secuencia natural. Se recomienda usar `tabindex="0"` solo cuando sea necesario y evitar valores positivos que alteren el flujo normal del documento.

Uso correcto de ARIA

ARIA (Accessible Rich Internet Applications) es un conjunto de atributos diseñado para mejorar la accesibilidad de interfaces dinámicas cuando el HTML semántico no es suficiente. Su función principal es comunicar a las tecnologías de asistencia el rol, estado y propiedades de los componentes interactivos. Por ejemplo, un menú desplegable creado con `div` puede adquirir significado semántico si se le asigna el rol `menu` y a sus elementos internos el rol `menuitem`.

Sin embargo, ARIA debe utilizarse con cuidado. La primera regla de ARIA es: “No usar ARIA si puedes usar HTML nativo”. Elementos como `button`, `input`, `select` o `nav` ya poseen semántica incorporada y son reconocidos por los lectores de pantalla sin necesidad de atributos adicionales. El abuso de ARIA puede generar conflictos si se asignan roles incorrectos o redundantes.

Entre los atributos más importantes se encuentran `aria-label` y `aria-labelledby`, que proporcionan nombres accesibles a los elementos; `aria-live`, que permite anunciar cambios dinámicos en regiones específicas de la página; y `aria-expanded`, que indica si un componente como un acordeón o menú está abierto o cerrado. El uso de `aria-live` es especialmente útil para notificar actualizaciones sin recargar la página, como mensajes de confirmación o alertas.

También es fundamental mantener sincronizados los estados visuales con los estados ARIA. Si un botón cambia a estado deshabilitado visualmente, debe reflejarse mediante `aria-disabled="true"` o el atributo `disabled`. De lo contrario, se produce una incongruencia entre lo que ve el usuario y lo que percibe el lector de pantalla.

Referencias

Mozilla Developer Network (MDN). (2024). *Document Object Model (DOM)*. Mozilla Foundation. https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model

Mozilla Developer Network (MDN). (2024). *ARIA: Accessible Rich Internet Applications*. Mozilla Foundation. <https://developer.mozilla.org/es/docs/Web/Accessibility/ARIA>

W3C. (2023). *Accessible Rich Internet Applications (WAI-ARIA) 1.2*. World Wide Web Consortium. <https://www.w3.org/TR/wai-aria-1.2/>

W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. World Wide Web Consortium. <https://www.w3.org/TR/WCAG22/>

Facebook Open Source. (2024). *React Documentation: Virtual DOM and Rendering*. Meta Platforms, Inc. <https://react.dev/learn/render-and-commit>

Google Developers. (2024). *Focus management and accessibility*. Google LLC. <https://developer.chrome.com/docs/lighthouse/accessibility/>

Deque Systems. (2024). *ARIA best practices and common mistakes*. Deque University. <https://dequeuniversity.com/rules/axe/4.8/aria-valid-attr-value>

Nielsen Norman Group. (2023). *Accessibility for Dynamic Content and Single Page Applications*. <https://www.nngroup.com/articles/aria-live/>

Angel Gaspar Reyes

Reporte de cambios — Tag Human Universal

- **Proyecto:** Tag Human Universal
- **Ruta:** c:\Users\garea\Desktop\Tag_Human_Universal>tag-human-universal
- **Fecha:** 2026-02-08

Resumen breve:

- Se aplicaron mejoras de accesibilidad en el frontend, específicamente en el componente que muestra el resultado del escaneo de QR.

- Se añadieron pruebas que validan atributos ARIA y comportamiento de foco.
- La suite de tests del frontend pasa correctamente.

Commits recientes (selección relevante):

- 0f9859d — 2026-02-08 — ANGELGASPAREYES: feat: accessible QR access result with focus, ARIA and tests
- ba1cd85 — 2026-01-27 — ANGELGASPAREYES: docs: Actualizar historias de usuario - Análisis de requerimientos
- 25bbb04 — 2026-01-23 — Lilia Hernandez Tun: feat: pipeline de CI integral para validacion y docker
- 45d3d94 — 2026-01-20 — OscarEL1: chore: configurar commitlint y husky para validar commits
- (Más historial disponible en el repositorio)

Cambios aplicados (detallado):

- [QRAccessResult.jsx](#):
 - [tabIndex](#) cambiado a valor numérico ([tabIndex={-1}](#)) para permitir foco programático sin incluir el elemento en el orden de tabulación.
 - Añadido `aria-atomic="true"` para garantizar que los lectores de pantalla anuncien el contenido completo al actualizarse.
 - Ajustado el efecto de [focus](#) en [useEffect](#) para que ocurra solo cuando [status](#) sea GRANTED o DENIED (evita foco en estado inicial "Esperando escaneo...").
- [QRAccessResult.test.jsx](#):
 - Añadido test que verifica `aria-atomic="true"` y que [tabIndex](#) es -1.
 - Tests existentes comprobantes: acceso concedido/denegado y presencia de [role="alert"](#) se mantienen.

Resultados de pruebas:

- Comando ejecutado: [npm test](#) (en frontend/)
- Resultado: Todos los tests pasan — 4 tests ejecutados y aprobados.

Estado del repositorio:

- No hay cambios sin commitear al momento de generar este reporte.

Impacto / motivos del cambio:

- Mejora de la experiencia para usuarios de tecnologías asistivas: los cambios aseguran que las actualizaciones dinámicas del DOM sean anunciadas correctamente y que el enfoque se gestione de forma no intrusiva.
- Mantiene buenas prácticas para live regions y gestión de foco en componentes dinámicos.